

บทที่ 3

หลักการเขียนโปรแกรม

3.1 CLDC

สภาพแวดล้อมการทำงานของจาวาใน MIDP มักมาพร้อมกับ K Virtual Machine (KVM) ไลบรารีของ CLDC และ MIDP และซอฟต์แวร์จัดการแอปพลิเคชัน (Application Management Software : AMS) ไลบรารีจาวาใน CLDC และ MIDP จะเป็นพื้นฐานรองรับการเขียนแอปพลิเคชันบนอุปกรณ์ไร้สาย ไลบรารีของ CLDC เป็นไลบรารีของเครือข่ายและระบบชั้นสูงที่ไม่จำกัดเฉพาะอุปกรณ์ใดอุปกรณ์หนึ่ง ประกอบด้วย คลาส 2 ประเภท คือ คลาสที่เป็นซับเซตของ J2SE และคลาสที่เกี่ยวกับการติดต่อสื่อสารทั่วไป

คลาสประเภทแรกของ CLDC เป็นซับคลาสของไลบรารี J2SE กำหนดไว้ในแพ็คเกจ java.lang java.util และ java.io โดยคลาสระบบและคลาสชนิดข้อมูลเข้ากันได้กับ J2SE และ J2EE และเพื่อให้เข้ากันได้ได้เคลื่อนย้ายข้ามแพลตฟอร์มได้สะดวก คลาสประเภทนี้จะใช้ชื่อคลาสและชื่อแพ็คเกจตามชื่อคลาสอย่างเดียวกันใน J2SE หรือตามชื่อซับคลาสใน J2SE ที่เกี่ยวข้องกัน โดยไม่มีการเพิ่มเมธีอด public หรือ protected หรือฟิลด์ซึ่งไม่มีในคลาสที่เกี่ยวข้องกันใน J2SE ความหมายของคลาสและเมธีอดจึงไม่เปลี่ยนไป

คลาสประเภทที่สองของ CLDC กำหนดไว้ในแพ็คเกจ javax.microedition.io คลาสเหล่านี้มีเฉพาะใน CLDC จึงไม่สามารถทำงานร่วมกับไลบรารีของ J2SE ได้ เป็นคลาสชั้นสูงเกี่ยวกับเรื่องทั่วไปของเครือข่าย กำหนดไว้ในแพ็คเกจ java.io และ java.net คลาสประเภทนี้จะเตรียมกรอบการติดต่อสื่อสารผ่านเครือข่ายให้กับอุปกรณ์ที่สนับสนุน J2ME โดยกรอบการสื่อสารนี้มักเรียกกันว่า Generic Connection Framework คลาสส่วนใหญ่ได้แก่อินเทอร์เน็ตเฟสที่แทนการสื่อสารรูปแบบต่างๆ เช่น ซ็อกเก็ต ดาต้าแกรม ซีเรียล และ http ขึ้นอยู่กับว่าผู้ผลิตอุปกรณ์หรือผู้ให้บริการเครือข่ายจะติดตั้งอินเทอร์เน็ตเฟสการเชื่อมต่อเหล่านี้เพียงบางตัวหรือครบทั้งชุด เมื่อพิจารณาจากความสามารถของอุปกรณ์และเครือข่าย

ข้อจำกัดของ CLDC CLDC และ KVM ออกแบบมาเพื่อใช้กับอุปกรณ์ที่มีทรัพยากรจำกัด จึงไม่สามารถสนับสนุนคุณสมบัติและฟังก์ชันได้หลากหลายเท่า J2SE คุณสมบัติบางอย่างที่มีใน J2SE ถูกดึงออกจาก CLDC และ KVM เพื่อลดขนาดและปรับปรุงประสิทธิภาพในการทำงาน โดยจะต้องศึกษาข้อจำกัดของ J2ME ก่อนที่จะเริ่มออกแบบและพัฒนาแอปพลิเคชัน

3.2 ประเภทของข้อมูลพื้นฐาน

CLDC สนับสนุนเฉพาะซับเซตของชนิดข้อมูลที้นำมาจาก J2SE ซึ่งได้แก่ byte short int long char และ boolean แต่ไม่สนับสนุนข้อมูลประเภท float และ double เนื่องจากสาเหตุสองประการ คือ อุปกรณ์เป้าหมายที่นำ CLDC ไปใช้ ส่วนใหญ่ไม่มีฮาร์ดแวร์รองรับตัวเลขทศนิยม และหากจะใช้ซอฟต์แวร์

เข้าช่วยก็จะต้องเสียค่าใช้จ่ายสูงมาก นอกจากนี้ CLDC ยังกำหนดคลาส type wrapper สำหรับข้อมูลทุกประการ ได้แก่

```
java.lang.Boolean
java.lang.Byte
java.lang.Character
java.lang.Integer
java.lang.Long
java.lang.Short
```

คลาส type wrapper กำหนดไว้ในจาวา เนื่องจากจาวามีระบบย่อยหลายระบบซึ่งทำงานได้เฉพาะกับอ็อบเจกต์เท่านั้นในกรณีนี้ คุณสามารถสร้างอ็อบเจกต์โดยใช้คลาส wrapper ซึ่งเก็บชนิดของข้อมูลที่นำมาจาก J2SE เอาไว้

3.3 ไลบรารีของ MIDP

ไลบรารีของ CLDC ช่วยสร้างฟังก์ชันที่ไม่จำกัดเฉพาะอุปกรณ์บางชนิด ไลบรารีของ MIDP กลับตรงกันข้ามได้แก่ ที่จัดการแอปพลิเคชันบนอุปกรณ์ ส่วนการติดต่อกราฟิกกับผู้ใช้ทั้งแบบพื้นฐานและซับซ้อน พื้นที่เก็บข้อมูลแบบคงตัว และความสามารถเพิ่มเติมด้านเครือข่าย

3.4 คลาส

คลาสโปรแกรมจัดการแอปพลิเคชัน เป็นคลาสที่ติดต่อกับโปรแกรมจัดการแอปพลิเคชันบนอุปกรณ์ถูกกำหนดไว้ในแพ็คเกจ `java.microedition.midlet` แอปพลิเคชันทั้งหลายที่เขียนใน MIDP จะต้องขยายคลาส `Midlet` ที่อยู่ในแพ็คเกจออกไป และจะต้องนำมธือดทั้ง 3 ซึ่งได้แก่ `startApp()` `pauseApp()` และ `destroyApp()` เข้ามาใช้

คลาสของส่วนติดต่อกราฟิกกับผู้ใช้ (GUI Class) เป็นชุดเครื่องมือ Abstract Windowing Toolkit ใน J2SE ออกแบบมาเพื่อใช้งานกับแอปพลิเคชันบนเครื่องคอมพิวเตอร์แบบตั้งโต๊ะและไม่สามารถนำมาใช้กับอุปกรณ์ไร้สายได้ เนื่องจากบริบททรัพยากรประเภทหน่วยความจำค่อนข้างมาก MIDP มีวิธีที่ต่างกันในการกำหนดไลบรารีสำหรับแพ็คเกจส่วนติดต่อกราฟิกกับผู้ใช้ ด้วยการใช้ API ขั้นสูงซึ่งเน้นความสามารถในการเคลื่อนย้ายข้ามอุปกรณ์ และ API ขั้นพื้นฐานซึ่งเน้นองค์ประกอบกราฟิกเฉพาะอุปกรณ์ และ input event ทั่วไป คลาสที่เชื่อมต่อกับส่วนติดต่อกราฟิกกับผู้ใช้ และ event-handling กำหนดไว้ในแพ็คเกจ `java.microedition.lcdui` Screen ถือเป็นซูเปอร์คลาสของคอมโพเนนต์ส่วนติดต่อกับผู้ใช้โดยใช้ API ขั้นสูง ประกอบไปด้วย Alert , Form , List , Textbox ฯลฯ Canvas และ Graphic เป็นคลาสหลักของ API ระดับต่ำ (Low Level) แอปพลิเคชันเกมส์เป็นแอปพลิเคชันที่ใช้ API ระดับต่ำในการสร้างส่วนติดต่อผู้ใช้

คลาสของพื้นที่เก็บข้อมูลแบบคงตัว ในบางครั้งแอปพลิเคชันที่เขียนบน MIDP จำเป็นต้องเก็บข้อมูลไว้บนอุปกรณ์อย่างถาวร คลาสที่กำหนดไว้ในแพ็คเกจ `java.microedition.rms` ให้กลไกการเก็บ

ข้อมูลถาวรที่เรียกว่า RecordStore โดยยอมให้แอปพลิเคชันเขียนลบและปรับปรุงบันทึกข้อมูลในหน่วยเก็บข้อมูลถาวรในอุปกรณ์

คลาสของเครือข่าย แม้ว่ากรอบการติดต่อสื่อสารทั่วไปที่กำหนดไว้ใน CLDC จะประกอบไปด้วยชุดของอินเทอร์เฟซการเชื่อมต่อเครือข่าย แต่ก็ไม่มีโปรโตคอลอยู่เบื้องหลังอินเทอร์เฟซการเชื่อมต่อจริงๆ หากแต่ปล่อยไว้ให้เป็นหน้าที่ของ MIDP ในบรรดาอินเทอร์เฟซการเชื่อมต่อเครือข่ายเหล่านี้ HttpURLConnection ถือเป็นอินเทอร์เฟซหลักที่ต้องมีใน MIDP เสมอ คลาสของอินเทอร์เฟซเหล่านี้กำหนดในแพ็คเกจ `java.microedition.io`

3.5 MIDlet

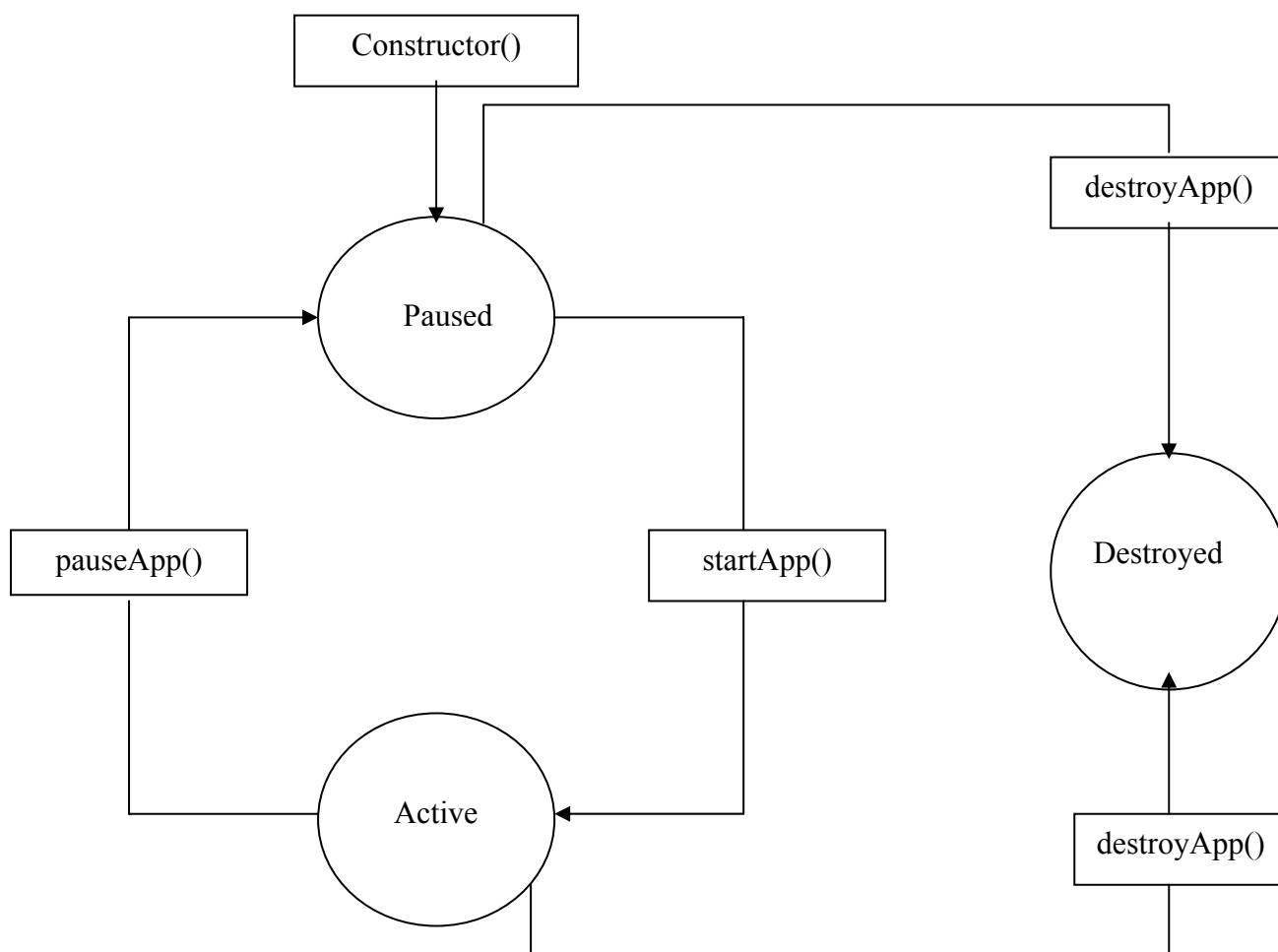
MIDlet หมายถึงแอปพลิเคชันบน MIDP MIDlet มีส่วนคล้ายกับจาวาแอปเพล็ต แม้จะไม่มีเมธอด `main()` แต่ MIDlet ก็นำคลาส `java.microedition.midlet.MIDlet` ตลอดจนเมธอด `startApp()` `pauseApp()` และ `destroyApp()` เข้ามาใช้ นอกจากนี้ MIDlet ยังกำหนด constructor แบบ `public` ที่ไม่มีอาร์กิวเมนต์ใดๆ อีกด้วย เราสามารถให้คำนิยามของคลาส `javax.microedition.midlet.MIDlet` ได้ดังนี้

```
Public abstract คลาส MIDlet extends Object {
    protected MIDlet()
    protected abstract void startApp() throws MIDletStateChangeException
    protected abstract void pauseApp()
    protected abstract void destroyApp(boolean unconditional)
        throws MIDletStateChangeException
    public final String getAppProperty(String key)
    public final void notifyDestroyed()
    public final void notifyPaused()
    public final String getAppProperty(String key)
    public final void resumeRequest()
}
```

คลาส MIDlet ระบุเมธอดที่สามารถเรียกใช้โดยซอฟต์แวร์จัดการแอปพลิเคชัน (AMS) เพื่อสั่งให้แอปพลิเคชัน MIDlet เริ่มต้นและหยุดทำงาน

3.5.1 วงจรการทำงานของ MIDlet การกระทำของ MIDlet ประกอบไปด้วย 3 สถานะ คือกำลังทำงาน หยุดชั่วคราวและถูกทำลาย อาศัยซอฟต์แวร์จัดการแอปพลิเคชัน เป็นตัวควบคุมการเปลี่ยนสถานะหนึ่งไปยังอีกสถานะหนึ่งควบคุมด้วยเมธอด `startApp()` `pauseApp()` และ `destroyApp()` ที่มาพร้อมกับ MIDlet จากรูปที่ 3.1 แสดงให้เห็นจุดเปลี่ยนระหว่างสถานะทั้ง 3 โดยการเรียกใช้เมธอดข้างต้น เมื่อ MIDlet พร้อมสั่งกระทำการ ซอฟต์แวร์จัดการแอปพลิเคชันจะสร้างตัวอย่าง MIDlet ขึ้นมาก่อนโดยใช้ constructor แบบ `public` ที่ไม่มีอาร์กิวเมนต์ใดๆ โดย MIDlet จะอยู่ในสถานะหยุดชั่วคราว จากนั้นซอฟต์แวร์จัดการแอปพลิเคชันจะเรียกเมธอด `startApp()` ขึ้นมา และ MIDlet จะเข้าสู่สถานะกำลังทำงาน

เปิดรับทรัพยากรที่ต้องการและเริ่มต้นการทำงาน ในสถานะนี้ MIDlet จะทำงานและดึงทรัพยากรที่ต้องการไว้ใช้งาน เมื่อซอฟต์แวร์จัดการแอปพลิเคชันไม่ต้องการให้ MIDlet ทำงานต่อไป ก็จะเรียกเมธอด `pauseApp()` จากนั้น MIDlet จะหยุดทำงานและเข้าสู่สถานะหยุดชั่วคราว ถิ่นทรัพยากรที่ดึงมาใช้งานและเข้าสู่ภาวะไม่ทำงาน MIDlet สามารถกลับไปอยู่ที่สถานะกำลังทำงาน ได้เมื่อซอฟต์แวร์จัดการแอปพลิเคชันเรียกเมธอด `startApp()` ขึ้นมา



รูป 3.1 วงจรการทำงานของ MIDlet

ท้ายที่สุดเมื่อซอฟต์แวร์จัดการแอปพลิเคชันไม่ต้องการเรียกใช้งาน MIDlet หรือต้องการเคลียร์หน่วยความจำเพื่อให้โปรแกรมอื่นได้ใช้งาน ก็จะส่งสัญญาณเตรียมทำลาย MIDlet ทั้งด้วยการเรียกเมธอด `destroyApp()` และเข้าสู่สถานะถูกทำลาย ซึ่ง MIDlet จะปล่อยทรัพยากรทั้งหมด ทำการจัดเก็บข้อมูลถาวรต่างๆ และหยุดการทำงานทั้งหมด หาก MIDlet อยู่ระหว่างสักระยะขั้นตอนสำคัญอยู่ ก็อาจร้องขอไม่เข้าสู่สถานะถูกทำลายได้โดยเรียกใช้ `MIDletStateChangeException` อย่างไรก็ตามซอฟต์แวร์จัดการแอปพลิเคชันอาจปฏิเสธหรือยินยอมตามคำร้องขอนี้ก็ได้ ตัวแปรบูลีน `unconditional` ในลายเซ็นของเมธอด `destroyApp()` จะเป็นตัวกำกับว่าการร้องขอนี้สมควรหรือไม่ ถ้าตัวบ่งชี้มีค่าเท่ากับ `true` การร้องขอ

เป็นอันตกไป ตรงกันข้ามหากมีค่าเท่ากับ false ก็จะได้รับคำตอบรับและจะเรียกเมธอด `destroyApp()` ขึ้นมาใหม่ในคราวต่อไป หาก `MIDlet` ต้องการเข้าสู่สถานะหยุดชั่วคราวหรือถูกทำลายด้วยตัวเอง ก็ สามารถทำได้โดยเรียกเมธอด `notifyPause()` หรือ `notifyDestroy()` ตามลำดับ โดยเมธอดทั้ง 2 จะแจ้งให้ ซอฟต์แวร์จัดการแอปพลิเคชันทราบว่า `MIDlet` ได้เข้าสู่สถานะหยุดชั่วคราว/ถูกทำลายแล้ว ในกรณีนี้ ซอฟต์แวร์จัดการแอปพลิเคชันจะไม่เรียกเมธอด `pauseApp()` หรือ `destroyApp()` อีก `MIDlet` จะเข้าสู่ สถานะหยุดทำงานได้ก็ต่อเมื่อผ่านสถานการณ์ทำงานมาก่อนแล้ว ในทางตรงกันข้าม หากต้องการเข้าสู่ สถานะถูกทำลาย สามารถเข้าได้โดยตรงจากสถานะหยุดทำงานชั่วคราวและสถานะกำลังทำงาน นอกจากนี้ยังสามารถเข้าสู่สถานะหยุดการทำงานได้ในขณะที่กำลังทำงาน หรือเมื่อได้รับคำสั่งจาก ซอฟต์แวร์จัดการแอปพลิเคชัน

3.6 ซอฟต์แวร์จัดการแอปพลิเคชัน

ซอฟต์แวร์แอปพลิเคชันที่มาพร้อมกับ MIDP ทำหน้าที่ควบคุมการติดตั้ง สิ่งกรทำการ และลบ `MIDlet` บางครั้งเราเรียกซอฟต์แวร์จัดการแอปพลิเคชันว่าเป็น ซอฟต์แวร์จัดการ `MIDlet` (`MIDlet Management Software`) หรือ โปรแกรมบริหารจาวาแอปพลิเคชัน (`Java Application Manager`)

3.7 ข้อจำกัด

3.7.1 ข้อจำกัดในการจัดการข้อผิดพลาด CLDC นั้นรองรับความผิดพลาดในการทำงานใน วงจำกัด โดยกำหนดคลาสแสดงข้อผิดพลาดไว้เพียง 2 คลาส คือ `java.lang.VirtualMachineError` และ `java.lang.OutOfMemoryError` ขณะที่คลาสดังกล่าวส่วนใหญ่ออกไป เนื่องจาก

3.7.1.1 ในระบบอุปกรณ์ฝังตัว การคืนสู่ภาวะทำงานผิดพลาด เป็นความสามารถเฉพาะบาง อุปกรณ์เท่านั้น ไม่ควรคาดหวังให้ผู้เขียนแอปพลิเคชันจัดการกับข้อบกพร่องเฉพาะอุปกรณ์เหล่านี้

3.7.1.2 ความผิดปกติประเภท `Error` มักไม่สามารถแก้ไขได้ การบรรจความสามารถในการ จัดการข้อผิดพลาดอย่างเต็มรูปแบบทำให้ต้องเสียค่าใช้จ่ายมาก และยังต้องกำหนดลำดับความสำคัญไว้สูง ทั้งที่อุปกรณ์ CLDC มีทรัพยากรค่อนข้างจำกัด

3.7.2 ข้อจำกัดของ KVM เนื่องจาก KVM เป็นเลเยอร์ `Java Virtual Machine` ที่รองรับไลบรารี ของ CLDC คุณสมบัติใดที่ CLDC ไม่สนับสนุน ก็จะถูกดึงออกจาก KVM ด้วยเช่นกัน อาทิ ตัวเลขทศนิยม `finalization` และการจัดการข้อผิดพลาดเฉพาะอุปกรณ์

3.7.2.1 ไม่สนับสนุน `Java Native Interface (JNI)` ด้วยสาเหตุ 2 ประการคือในแบบจำลองความ ปลอดภัยของ CLDC ผู้เขียนแอปพลิเคชันไม่สามารถดาวน์โหลดไลบรารีใหม่ๆที่ยังมีฟังก์ชันเดิมของ อุปกรณ์อยู่ และไม่สามารถเข้าถึงฟังก์ชันเดิมของอุปกรณ์ที่ไม่มีในไลบรารีของจาวาได้ นอกจากนี้ การนำ `JNI` มาใช้ยังเพิ่มภาระให้กับหน่วยความจำบนอุปกรณ์ CLDC

3.7.2.2 ไม่สนับสนุนโปรแกรมบรรจุคลาสที่ผู้ใช้งานกำหนดขึ้นเอง และโปรแกรมบรรจุคลาสใน KVM ก็ไม่อนุญาตให้ผู้ใช้งานลบล้าง เขียนทับ หรือกำหนดค่าใหม่ด้วย

3.7.2.3 ไม่สนับสนุน Reflection , RMI และ การเรียกใช้ลำดับอ็อบเจกต์ ทำให้โปรแกรม CLDC ไม่สามารถตรวจสอบเนื้อหาในคลาสอ็อบเจกต์ เมธอด ฯลฯ

3.7.2.4 ไม่สนับสนุน Thread Group หรือ Daemon คือ KVM สนับสนุนแอปพลิเคชันแบบทำงานคู่ขนาน (Multi-Thread) แต่ไม่รองรับกลุ่มงานย่อย (Thread Object) หรืองานเบื้องหลัง (Daemon Thread)

3.8 ระบบความปลอดภัยใน J2ME

วัตถุประสงค์สำคัญของการรักษาความปลอดภัยบนระบบคอมพิวเตอร์คือ การป้องกันทรัพยากรของระบบจากการเจาะเข้ามาทั้งที่ไม่ประสงค์ดีและไม่ตั้งใจ การดาวน์โหลดเนื้อหาและแอปพลิเคชันตามข้อกำหนดของ CLDC ระบุไว้ ทำให้การรักษาความปลอดภัยบนระบบเครือข่ายทวีความสำคัญมากขึ้น จาวานั้นออกแบบมาโดยคำนึงถึงความปลอดภัยมาตั้งแต่ต้น โดย J2SE เองมีคุณสมบัติด้านความปลอดภัยที่หลากหลาย เช่น การตรวจไบต์โค้ด และโปรแกรมจัดการความปลอดภัย อย่างไรก็ตาม เนื่องจากรหัสที่ J2SE ใช้ในการรักษาความปลอดภัยมีขนาดใหญ่เกินกว่าหน่วยความจำของอุปกรณ์ไร้สายที่มีทรัพยากรจำกัด จึงต้องลดทอนคุณสมบัติดังกล่าวลงให้เหมาะสมกับความต้องการระบบของอุปกรณ์ไร้สาย

3.9 การเวอร์ิฟายและฟรีเวอร์ิฟายไฟล์คลาส

เนื่องจากทรัพยากรที่มีจำกัดบนอุปกรณ์ไร้สายการเวอร์ิฟายคลาสของ KVM จึงต้องแยกกันกระทำ โดยบางส่วนกระทำนอกตัวอุปกรณ์นั้น เรียกว่า ฟรีเวอร์ิฟิเคชัน

3.10 แบบจำลอง Sand Box

J2ME ได้นำแบบจำลองรักษาความปลอดภัยมาจาก J2SE หลักการของแบบจำลอง Sand Box ใน J2ME คือ ให้แอปพลิเคชันที่เขียนด้วยจาวาทำงานในสภาพแวดล้อมปิด ซึ่งแอปพลิเคชันสามารถเข้าถึงเฉพาะ API ที่กำหนดโดยคอนฟิเจอร์ชัน โพรไฟล์ และคลาสเปิดที่อุปกรณ์นั้นได้รับอนุญาตให้ใช้นอกจากนี้ แบบจำลอง Sand Box ยังหมายถึง

1. จาวาคลาสได้รับเวอร์ิฟายและรับรองแล้วว่าเป็นจาวาแอปพลิเคชันที่ถูกต้อง
2. นักพัฒนาแอปพลิเคชันมีชุดจาวา API จำนวนจำกัดซึ่งกำหนดไว้ล่วงหน้าโดยโพรไฟล์ CLDC และคลาสเปิดที่ได้รับอนุญาตให้ใช้
3. การดาวน์โหลดและการจัดการจาวาแอปพลิเคชันบนอุปกรณ์ เกิดขึ้นในระดับโค้ดพื้นฐานที่อยู่ภายใน Virtual Machine และไม่มีโปรแกรมบรรจุคลาสที่ผู้ใช้งานกำหนดขึ้นเอง เพื่อป้องกันไม่ให้ผู้เขียนโปรแกรมเขียนทับกลไกมาตรฐานการโหลดคลาสของ Virtual Machine
4. ชุดฟังก์ชันเดิมของอุปกรณ์ที่เข้าถึง Virtual Machine ถูกปิดการใช้งาน ซึ่งหมายความว่านักพัฒนาแอปพลิเคชันไม่สามารถดาวน์โหลดไลบรารีใหม่ๆ ที่มีฟังก์ชันเดิมของอุปกรณ์มาใช้ได้ หรือไม่สามารถเข้าถึงฟังก์ชันเดิมของอุปกรณ์ซึ่งไม่มีในไลบรารีของจาวาที่กำหนดโดย CLDC Profile หรือคลาสเปิดที่ได้รับอนุญาตให้ใช้